

human

Show me natural areas in Exeter

host_agent

Tool generate_metadata_for_all_artifacts with args : {}

Tool generate_metadata_for_all_artifacts returned result : {}

Tool send_message with args : {'target': 'named_area', 'task_description': 'Find the named area polygon for Exeter'}

named_area_agent

Tool call_os_ngd with args : {'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_named_area'}

Tool call_os_ngd returned result : ['Search results have been found. Artifact generated is : exeter_named_area, Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the named area polygon for Exeter., count of records fetched : 98.',]

Tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_named_area' to find the specific named area polygon for Exeter. Save the filtered results as 'filtered_exeter_named_area'."}

data_analysis_agent

Tool generate_metadata_for_artifacts with args : {'artifact_names': ['exeter_named_area']}

Tool generate_metadata_for_artifacts returned result : [...metadata summary...]

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def filter_exeter_named_area(data: list):
    gdf = data[0]
    # Filter for rows where the main name is 'Exeter' in any of the name columns
    # and where the description is relevant to a settlement/city/town
```

```

name_columns = ['name1_text', 'name2_text', 'name3_text', 'name4_text']
settlement_descriptions = ['City', 'Town', 'Part Of Settlement', 'Suburban Area', 'Village']
mask = (
    (gdf[name_columns].apply(lambda row: row.astype(str).str.lower().str.strip().eq('exeter').any(), axis=1)) &
    (gdf['description'].isin(settlement_descriptions))
)
filtered = gdf[mask]
summary = f"Filtered named area polygons for Exeter with settlement-related descriptions. Number of entries: {len(filtered)}."
artifact_name = 'filtered_exeter_named_area'
artifact_description = 'GeoDataFrame containing only the named area polygon(s) for Exeter, filtered by name and settlement-related descriptions.'
return [summary, artifact_name, artifact_description, filtered]

filter_exeter_named_area(data)

```

Tool Code Executor

```

import matplotlib
matplotlib.use('Agg')
def filter_exeter_named_area(data: list):
    gdf = data[0]
    # Filter for rows where the main name is 'Exeter' in any of the name columns
    # and where the description is relevant to a settlement/city/town
    name_columns = ['name1_text', 'name2_text', 'name3_text', 'name4_text']
    settlement_descriptions = ['City', 'Town', 'Part Of Settlement', 'Suburban Area', 'Village']
    mask = (
        (gdf[name_columns].apply(lambda row: row.astype(str).str.lower().str.strip().eq('exeter').any(), axis=1)) &
        (gdf['description'].isin(settlement_descriptions))
    )
    filtered = gdf[mask]
    summary = f"Filtered named area polygons for Exeter with settlement-related descriptions. Number of entries: {len(filtered)}."
    artifact_name = 'filtered_exeter_named_area'
    artifact_description = 'GeoDataFrame containing only the named area polygon(s) for Exeter, filtered by name and settlement-related descriptions.'
    return [summary, artifact_name, artifact_description, filtered]

```

Filtered named area polygons for Exeter with settlement-related descriptions. Number of entries: 1. An artifact has been generated with the name: filtered_exeter_named_area and the description: GeoDataFrame containing only the named area polygon(s) for Exeter, filtered by name and settlement-related descriptions.

named_area_agent

The named area polygon for Exeter has been successfully filtered and saved. The artifact is named filtered_exeter_named_area, containing only the relevant polygon(s) for Exeter.

Tool send_message with args : {'target': 'land_features', 'task_description': "Find natural land features (such as coniferous trees, cliffs, heath, rough grassland, etc.) within the area defined by the artifact 'filtered_exeter_named_area'."}

land_features_agent

Tool call_os_ngd with args : {'filters': ['Coniferous Trees', 'Cliff', 'Heath', 'Rough Grassland'], 'bbox': 'filtered_exeter_named_area', 'polygon_or_point': True, 'street_address': False, 'filename': 'natural_land_features_exeter'}

Tool call_os_ngd returned result : ['Multiple search results have been found in multiple datasets. A summary of each is provided. Artifacts generated are :
['land_natural_land_features_exeter', 'landform_natural_land_features_exeter'], Descriptions are : ["A geopandas dataframe containing land data with filters and bbox applied for the query Find natural land features (such as coniferous trees, cliffs, heath, rough grassland, etc.) within the area defined by the artifact 'filtered_exeter_named_area'. using filters ['Coniferous Trees', 'Cliff', 'Heath', 'Rough Grassland']. ", "A geopandas dataframe containing land form data with filters and bbox applied for the query Find natural land features (such as coniferous trees, cliffs, heath, rough grassland, etc.) within the area defined by the artifact 'filtered_exeter_named_area'. using filters ['Coniferous Trees', 'Cliff', 'Heath', 'Rough Grassland']."], counts of records fetched are : [18, 19].', [,]]

Tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Please analyze the artifacts 'land_natural_land_features_exeter' and 'landform_natural_land_features_exeter' to provide insights on the natural land features such as coniferous trees, cliffs, heath, and rough grassland within the area defined by 'filtered_exeter_named_area'. Save the filtered results as 'analyzed_natural_land_features_exeter'."}

data_analysis_agent

Tool generate_metadata_for_artifacts with args : {'artifact_names': ['land_natural_land_features_exeter', 'landform_natural_land_features_exeter', 'filtered_exeter_named_area']}

Tool generate_metadata_for_artifacts returned result : [...metadata summary...]

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def analyze_natural_land_features_exeter(data):
    import geopandas as gpd
    import pandas as pd
    # Unpack data
```

```

land_natural = data[0]
landform_natural = data[1]
exeter_area = data[2]
# Ensure CRS matches for spatial join
if land_natural.crs != exeter_area.crs:
    land_natural = land_natural.to_crs(exeter_area.crs)
if landform_natural.crs != exeter_area.crs:
    landform_natural = landform_natural.to_crs(exeter_area.crs)
# Filter for relevant features in both datasets
land_natural_filtered = land_natural[
    (
        land_natural['description'].str.contains('Coniferous Trees', case=False, na=False) |
        land_natural['description'].str.contains('Heath', case=False, na=False) |
        land_natural['description'].str.contains('Rough Grassland', case=False, na=False) |
        land_natural['oslandcovertierb'].str.contains('Coniferous Trees', case=False, na=False) |
        land_natural['oslandcovertierb'].str.contains('Heath', case=False, na=False) |
        land_natural['oslandcovertierb'].str.contains('Rough Grassland', case=False, na=False)
    )
]
landform_natural_filtered = landform_natural[
    landform_natural['description'].str.contains('Cliff', case=False, na=False)
]
# Spatial filter: only features within the Exeter named area
exeter_geom = exeter_area.iloc[0].geometry
land_natural_filtered = land_natural_filtered[land_natural_filtered.intersects(exeter_geom)]
landform_natural_filtered = landform_natural_filtered[landform_natural_filtered.intersects(exeter_geom)]
# Combine results
combined = gpd.GeoDataFrame(
    pd.concat([land_natural_filtered, landform_natural_filtered], ignore_index=True),
    crs=land_natural_filtered.crs if not land_natural_filtered.empty else landform_natural_filtered.crs
)
summary = (
    f"Filtered natural land features in Exeter: "
    f"{len(land_natural_filtered)} from land_natural_land_features_exeter, "
    f"{len(landform_natural_filtered)} from landform_natural_land_features_exeter. "
    f"Total: {len(combined)} features."
)
description = (
    "GeoDataFrame containing all coniferous trees, heath, rough grassland, and cliff features "
    "within the Exeter named area, from both land_natural_land_features_exeter and "
    "landform_natural_land_features_exeter datasets."
)
return [summary, 'analyzed_natural_land_features_exeter', description, combined]

```

Filtered natural land features in Exeter: 18 from land_natural_land_features_exeter, 19 from landform_natural_land_features_exeter. Total: 37 features.

An artifact has been generated with the name: analyzed_natural_land_features_exeter. This GeoDataFrame contains all coniferous trees, heath, rough grassland, and cliff features within the Exeter named area, sourced from both the land_natural_land_features_exeter and landform_natural_land_features_exeter datasets.

land_features_agent

The search for natural land features within the 'filtered_exeter_named_area' has been completed. Here are the results:

- **Artifacts Generated:**
- land_natural_land_features_exeter: Contains 18 records of natural land features.
- landform_natural_land_features_exeter: Contains 19 records of natural land features.

Analysis Result:

- A combined analysis of both datasets resulted in a total of 37 natural land features, including coniferous trees, cliffs, heath, and rough grassland.
- The analyzed data is saved as analyzed_natural_land_features_exeter.

These artifacts provide a comprehensive view of the natural land features within the specified area.

Tool send_message with args : {'target': 'plotting_agent', 'task_description': "Plot the natural land features in Exeter using the artifact 'analyzed_natural_land_features_exeter'. Add a boundary for the Exeter area using 'filtered_exeter_named_area'."}

plotting_agent

Tool generate_metadata_for_artifacts with args : {'artifact_names': ['analyzed_natural_land_features_exeter', 'filtered_exeter_named_area']}

Tool generate_metadata_for_artifacts returned result : [...metadata summary...]

Tool Code Executor

```
import matplotlib
matplotlib.use('Agg')
def plot_natural_land_features_with_boundary(data: list):
    import geopandas as gpd
    import folium
    from shapely.geometry import mapping
    import os
```

```

# Unpack data
gdf_natural = data[0].copy()
gdf_boundary = data[1].copy()

# Ensure CRS is EPSG:4326 for folium
if gdf_natural.crs is None or gdf_natural.crs.to_epsg() != 4326:
    gdf_natural = gdf_natural.to_crs(epsg=4326)
if gdf_boundary.crs is None or gdf_boundary.crs.to_epsg() != 4326:
    gdf_boundary = gdf_boundary.to_crs(epsg=4326)

# Get centroid of Exeter for map initialization
centroid = gdf_boundary.geometry.iloc[0].centroid
m = folium.Map(location=[centroid.y, centroid.x], zoom_start=13, tiles='cartodbpositron')

# Plot Exeter boundary
folium.GeoJson(
    gdf_boundary.geometry.iloc[0],
    name='Exeter Boundary',
    style_function=lambda x: {'color': 'blue', 'weight': 3, 'fillOpacity': 0.05}
).add_to(m)

# Plot each natural land feature
for idx, row in gdf_natural.iterrows():
    desc = row['description'] if 'description' in row and row['description'] else 'Natural Feature'
    folium.GeoJson(
        row['geometry'],
        name=f"{desc}",
        style_function=lambda x: {'color': 'green', 'weight': 2, 'fillOpacity': 0.3},
        tooltip=desc
    ).add_to(m)

# Add layer control
folium.LayerControl().add_to(m)

# Save map
map_filename = 'natural_land_features_exeter_map.html'
m.save(map_filename)

# Prepare summary
preview = gdf_natural[['description', 'geometry_area_m2']].head(5)
summary = f"Plotted {len(gdf_natural)} natural land features in Exeter. First 5: {preview.to_dict(orient='records')}}"
artifact_name = 'natural_land_features_exeter_map'
artifact_description = 'Folium map showing natural land features in Exeter with the city boundary.'

return [summary, artifact_name, artifact_description, map_filename]

```


Plotted 37 natural land features in Exeter. First 5: [{description: 'Coniferous Trees', geometry_area_m2: 1451.412}, {description: 'Coniferous Trees', geometry_area_m2: 1148.307}, {description: 'Coniferous Trees', geometry_area_m2: 388.76}, {description: 'Coniferous Trees', geometry_area_m2: 507.085}, {description: 'Coniferous Trees', geometry_area_m2: 1534.429}]

Artifact name: natural_land_features_exeter_map

Artifact description: Folium map showing natural land features in Exeter with the city boundary.

Artifact data: natural_land_features_exeter_map.html

host_agent

I have found and mapped the natural areas in Exeter, including features such as coniferous trees, heath, rough grassland, and cliffs. The map includes the boundary of Exeter and highlights 37 natural land features within the city.

If you would like to view or download the map, let me know! If you need more details about specific types of natural areas or want to focus on a particular part of Exeter, please specify.